

Timeline Check-in - w/ Revisions

Madeline, Audrey, Maisie

2026-05-27

1. Set up

Packages

```
# general use
library(tidyverse)
library(janitor)

# file organization
library(here)

# Boxplot Visualization

library(hrbrthemes)

library(viridis)

# text label instead of legend
library(cowplot)

# Ridgeline Plot
#install.packages("ggribes")
library(ggribes)
#install.packages("ggplot2")
library(ggplot2)

# Color Palette
library(NatParksPalettes)
library(wesanderson)
```

```
# Use ragg_png device for better font rendering and system font support
knitr::opts_chunk$set(dev = "ragg_png")

# to stack plots
library("patchwork")
```

Data

```
# weather data from NOAA weather station
weather <- read_csv(here("data", "NOAA-weather-data.csv"))

# water quality survey metadata
metadata <- read_csv(here("data", "NCOS_YSI_Water_Quality_Monitoring_0.csv"))

# water quality parameter measurements
water_parameters <- read_csv(here("data", "YSI_Data_Begin_1.csv"))
```

Cleaning Code

Cleaning Weather

```
weather_clean <- weather |>
  # made column names all lowercase
  clean_names() |>
  # puts date in proper format and makes sure its understood as
  # date (not as chr)
  mutate(date = mdy(date)) |>
  # keeping only these 4 columns of interest
  select(date, prcp, tmax, tmin) |>
  # creating new columns for month and year
  # extracting month from the date (needs to be in the standard date format
  # for this function to work)
  mutate(month = month(date),
         year = year(date)) |>
  mutate(wy = case_when(
    # when month is october or later, assign a value of year +1
    # in the water year for next year
    month >= 10 ~ year + 1,
```

```

# keeping all months < 10 the same (sept or earlier), assigning value
# of existing year
.default = year
))

```

Cleaning Metadata

```

metadata_clean <- metadata |>
# col names easier to work with (all lowercase w/ _)
clean_names() |>
# "!" is the "not-operator", columns we DONT want to keep, excluding
select(!c(object_id, creator, editor)) |>
mutate(monitoring_date = mdy_hms(monitoring_date)) |>
rename(monitoring_datetime = monitoring_date) |>
mutate(month = month(monitoring_datetime),
       year = year(monitoring_datetime)) |>
# assigning water year
mutate(wy = case_when(
  month >= 10 ~ year + 1,
  .default = year )) |>
# creatig new column for site full name
mutate(site_full = case_when(
  site_name == "east_channel" ~ "East Channel",
  site_name == "phelps_bridge" ~ "Phelps Bridge",
  site_name == "venoco_bridge" ~ "Venoco Bridge",
  site_name == "other" ~ "Other"
)) |>
# filter out Other observations
filter(site_full != "Other")

```

Cleaning Water Parameters

```

# creating a new object from water parameters
water_parameters_clean <- water_parameters |>
# cleaning column names
clean_names() |>
# "!" not operator, excluding unneeded columns
select(!c(creator, editor)) |>

```

```

# renaming depth code to be elevation code to better represent methodology
rename(elevation_code = depth_code) |>
## Everything below came from visualizing the data first and defining what
## to fix/add
# fixing a mistake in the data
mutate(elevation_code = case_when(
  # anytime theres a 10 in elevation code, replace it with a 1
  elevation_code == 10 ~ 1,
  # everything else, keep the same
  .default = elevation_code)) |>
mutate(
  # turning column into a factor
  elevation_code = as_factor(elevation_code),
  # defining the correct order for the levels of the factor
  elevation_code = fct_relevel(elevation_code,
                                # write in the order you want
                                "0", "1", "2", "3", "4", "5", "6"))

```

Joining Code

Water Quality = Metadata + Water Parameters

```

# full joining
water_quality <- full_join(
  # left data frame
  x = metadata_clean,
  # right data frame
  y = water_parameters_clean,
  # name the columns in each df that you want to join by
  # (naming the shared column)
  by = c("global_id" = "parent_global_id")) |>
select(
  # information about the observation
  global_id, monitoring_datetime, time_of_wq_measurement, monitor_name,
  weather,
  site_name, other_site_name, site_full, x, y,
  # water information
  water_surface_elevation, flow, notes,
  # measurement information
  elevation_code, do_mg_l, do_percent_sat, conductivity_u_s_cm,

```

```

    salinity_ppt, temperature_c
  ) |>
  # taking the date out of this column, dont want the time
  mutate(date = date(monitored_datetime)) |>
  # moving the new date column to a better location within the df
  relocate(date, .after = monitored_datetime) |>
  # dont need this column anymore
  select(!monitored_datetime)

```

Water Quality Weather = Water Quality + Weather

```

# left join
water_quality_weather <- left_join(
  # left data frame
  x = water_quality,
  # right data frame
  y = weather_clean,
  # joining these dfs by their date column
  by = "date")

```

Wrangling/Summarizing Code

Salinity Outliers

```

# creating new obj
salinity_outlier_ids <- water_quality_weather |>
  # filtering to only include outlier values
  filter(salinity_ppt > 1410) |>
  # extracting the global id column as a vector using pull
  pull(global_id)

```

Summarizing Precipitation

```

# create new object from `water_quality_weather`
monthly_prpc <- water_quality_weather |>
  # only include elevation codes 0, 1, and 2

```

```

filter(elevation_code %in% c("0", "1", "2")) |>
# get rid of site names w/ "other"
filter(!site_full == "NA") |>
# removing any duplicate rows based on the combination of month,
# year, and wy
distinct(month, year, prcp, site_full) |>
# grouping by month and year
group_by(month, year) |>
# calculating the sum of prcp per month (not per site as it is a weather
# event and not site specific)
summarise(sum_prcp = sum(prcp, na.rm = TRUE)) |>
# ungrouping
ungroup() |>
# creating a date column to be read as a date
mutate(date = lubridate::make_date(.data$year, .data$month, 1)) |>
# creating levels to `month`, putting them in a logical order
mutate(month = factor(month,
                      levels = 1:12,
                      labels = c("Jan", "Feb", "Mar", "Apr", "May", "Jun",
                                "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"),
                      ordered = TRUE))

```

Filtering and Summarizing Salinity

```

salinity_filtered <- water_quality_weather |>
# keeping only elevation codes 0, 1, and 2
filter(elevation_code %in% c("0", "1", "2")) |>
# filtering the column global_id to exclude the values that match those
# in `salinity_outlier_ids`
filter(!global_id %in% salinity_outlier_ids) |>
# dropping the column for other_site_names
select(!other_site_name) |>
# get rid of site names w/ "other"
filter(!site_name == "other") |>
# setting the order of the levels (from lowest to highest avg salinity
# value)
# helps the order of the visualizations
mutate(site_full = fct_relevel(site_full,
                              "Phelps Bridge",
                              "East Channel",

```

```

                                "Venoco Bridge")) |>
# taking out sites w/ NA
filter(!site_full == "NA") |>
# keeping only columns of interest (to reduce confusion)
select(site_full, water_surface_elevation, elevation_code, salinity_ppt,
       temperature_c, month, year, wy) |>
# creating levels to `month`, putting them in a logical order
mutate(month = factor(month,
                      levels = 1:12,
                      labels = c("Jan", "Feb", "Mar", "Apr", "May",
                                "Jun", "Jul", "Aug", "Sep", "Oct",
                                "Nov", "Dec"),
                      ordered = TRUE)) |>
# group by site full, month, and year
group_by(site_full, month, year) |>
# calculate mean salinity (per site per month)
summarize(mean_salinity = mean(salinity_ppt, na.rm = TRUE),
          .groups = "drop")

```

Joining Monthly_prdp and Salinity_filtered

```

# creating new object ~salinity_prdp`
salinity_prdp <- salinity_filtered |>
# joining `monthly_prdp` to `salinity_filtered` by the columns
# month and year
left_join(monthly_prdp, by = c("month", "year"))

```

Exploratory Figure Code

Exploring Salinity

```

# create new object `exploratory_salinity` from `water_quality_weather`
exploratory_salinity <- water_quality_weather |>
# only keep elevation codes 0, 1, 2, and 3 (keeping
# three just for visualization)
filter(elevation_code %in% c("0", "1", "2", "3")) |>
# filtering the column global_id to exclude the values that match
# those in `salinity_outlier_ids`

```

```

filter(!global_id %in% salinity_outlier_ids) |>
# dropping the column `other_site_names`
select(!other_site_name) |>
# filterign out site names that match "other"
filter(!site_name == "other") |>
# setting the order of the levels (from lowest to highest avg
# salinity value)
# helps the order of the visualizations
mutate(site_full = fct_relevel(site_full,
                              "Phelps Bridge",
                              "East Channel",
                              "Venoco Bridge")) |>

# taking out sites w/ NA
filter(!site_full == "NA") |>
# keeping only columns of interest (to reduce confusion)
select(site_full, water_surface_elevation, elevation_code, salinity_ppt,
       temperature_c, month, year) |>
# group by site, elevation code, month, and year
group_by(site_full, elevation_code, month, year) |>
# calculate mean salinity
summarize(mean_salinity = mean(salinity_ppt, na.rm = TRUE),
          .groups = "drop") |>
# creating levels to `month`, putting them in a logical order
mutate(month = factor(month,
                      levels = 1:12,
                      labels = c("Jan", "Feb", "Mar", "Apr", "May",
                                "Jun", "Jul", "Aug", "Sep", "Oct",
                                "Nov", "Dec"),
                      ordered = TRUE))

```

Exploring Water Surface Elevation

```

# create a new object `monthly_surface_elv` from `water_quality_weather`
monthly_surfcae_elv <- water_quality_weather |>
# only include elevation codes 0, 1, and 2
filter(elevation_code %in% c("0", "1", "2")) |>
# Filter out site names matchign to "other"
filter(!site_name == "other") |>
# group by site, month, and year
group_by(site_full, month, year) |>

```



```

# calculate mean water surface elevation
summarise(avg_elv = mean(water_surface_elevation, na.rm = TRUE)) |>
# ungroup data frame
ungroup() |>
# create a date column to be read as a date
mutate(date = lubridate::make_date(.data$year, .data$month, 1)) |>
# creating levels to `month`, putting them in a logical order
mutate(month = factor(month,
                      levels = 1:12,
                      labels = c("Jan", "Feb", "Mar", "Apr", "May",
                                "Jun", "Jul", "Aug", "Sep", "Oct",
                                "Nov", "Dec"),
                      ordered = TRUE))

## Joining data frames
elv_viz <- monthly_surfcae_elv |>
# joining by columns month, year, and site
left_join(salinity_prctp, by = c("month", "year", "site_full")) |>
# filtering out all NA average elevation values
filter(!avg_elv == "NaN")

```

Creating Precipitation Peaks

```

# creating a new object `precipitation_peaks` from `monthly_prctp`
precipitation_peaks <- monthly_prctp |>
# including the dates of the top 4 monthly precipitation
filter(date %in% c("2023-03-01",
                  "2021-01-01",
                  "2023-02-01",
                  "2021-10-01")) |>
# ungrouping data frame
ungroup() |>
# keeping these 4 distinct dates
distinct(date, .keep_all = TRUE) |>
# adding a label to each event
mutate(label = c("Rain Event (6.05 in)",
                  "Rain Event (5.40 in)",
                  "Rain Event (6.18 in)",
                  "Rain Event (3.45 in)"),
# setting the y position for each event

```

```

    label_y = c(6.5, 6.5, 8, 6.25)) |>
# sort chronologically (better legend appearance)
arrange(date) |>
# lock levels in that order
mutate(label = factor(label, levels = label))

# creating a new object `precipitation_peaks_facet` from `precipitation_peaks`
precipitation_peaks_facet <- precipitation_peaks |>
# duplicating all 4 peaks across each unique site in salinity_prdp
cross_join(tibble(site_full = unique(salinity_prdp$site_full))) |>
# assigning y position for each peak label by date
mutate(label_y_2 = case_when(
  # y position for January 2021 rain event
  date == "2021-01-01" ~ 40,
  # y position for February 2023 rain event
  date == "2023-02-01" ~ 35,
  # y position for March 2023 rain event
  date == "2023-03-01" ~ 45,
  # y position for October 2021 rain event
  date == "2021-10-01" ~ 50))

```

2. Ideas for analysis and background

The overarching question for our project is: How does precipitation explain salinity?

We expect that more precipitation will generally lead to lower salinity because rainfall adds freshwater and dilutes saltier water. However, salinity is also affected by water depth, evaporation and whether the ecosystem is connected to the ocean.

This is especially relevant as North Campus Open Space is a connected coastal wetland system. Because of this, the salinity is influenced by more than rainfall alone. Harrington and Wood (2014), although focused on an Australian wetland, help support this idea by showing that ocean-driven saltwater movement can also affect salinity. Rainfall can lower salinity by adding freshwater, while sea-level changes and saltwater entering the system can raise salinity again.

Additionally, between 2015 to 2016 salinity was reported to increase at NCOS during dry periods because evaporation concentrates salts, while rainfall can lower salinity by adding freshwater (Clark, 2016). Rainfall lowered salinity by adding freshwater to the slough, but major storm and breach events reconnected the slough to the ocean, allowing seawater to enter and raise salinity again (Clark, 2016). The report also notes that salinity often increases with depth, so precipitation helps explain salinity but is not the only factor (Clark, 2016).

References:

Clark, R. (2016). Water Quality of North Campus Open Space & Devereux Slough: Fall 2015 - Spring 2016. [online] Available at: <https://escholarship.org/uc/item/2923f039> [Accessed 20 May 2026].

Wood, C. and Harrington, G.A. (2014). Influence of Seasonal Variations in Sea Level on the Salinity Regime of a Coastal Groundwater-Fed Wetland. *Groundwater*, 53(1), pp.90–98. doi:10.1111/gwat.12168.

Ideas for analyses to answer question and some preliminary code to run those analyses:

To answer our question, “How does precipitation explain salinity?”, we analysed whether monthly precipitation is related to monthly average salinity across NCOS sites. Our draft code already cleans and joins the weather, metadata, and water-quality datasets, then calculates total monthly precipitation and mean monthly salinity by site.

We also chose to use Spearman’s correlation coefficient to test whether salinity tends to decrease as precipitation increases.

The test gave a coefficient value of 0.0166 and p-value of 0.8251, suggesting that precipitation alone does not strongly explain salinity.

3. Visualizations directly relevant to answering questions

Figures

Figure 1. - Precipitation and Salinity Through Time

```
# Top half of Figure 1
# base layer: ggplot
# data: salinity_prdp
prcp_plot <- ggplot(data = monthly_prdp,
  # x axis: date
  mapping = aes(x = date,
    # y axis: monthly precipitation sum
    y = sum_prdp)) +
  # prdp displayed by columns
  geom_col(color = "darkblue") +
  # adding themeing
  theme_bw() +
  # x-axis breaks every 6 months (to reduce cognitive load)
  # Format date axis labels as abbreviated month and full year
```

```

scale_x_date(date_breaks = "6 months", date_labels = "%b %Y") +
# angling labels to 45 degrees and to be right-aligned
theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
# expanding limits to make room for text
scale_y_continuous(limits = c(0, 8)) +
# labeling x and y axes
labs(x = "Date",
      y = "Total Monthly Precipitation (in)",
      # adding title
      title = "Total Monthly Precipitation at NCOS") +
# removing legend, redundant
theme(legend.position = "none") +
# adding in lines for prcp events
geom_vline(data = precipitation_peaks,
           # centering them on dates
           aes(xintercept = date,
              # coloring them by their established label
              color = label),
           # signified by dotted lines
           linetype = "dotted",
           # width of lines
           linewidth = 1.2) +
# adding custom color palette
scale_color_manual(values = wes_palette("AsteroidCity1")) +
# removing space between data and x-axis
scale_y_continuous(expand = c(0,0.1)) +
# adding labels to each dotted line from the `precipitation_peaks` object
geom_text(data = precipitation_peaks,
          # x-axis location: date
          # y-axis location: set y value from earlier chunk
          # label = the defined label from earlier chunk
          # colro by label
          aes(x = date, y = label_y, label = label, color = label),
          # 45 degree angle for readability
          angle = 45,
          # slightly to right of vertical dotted line
          hjust = 1.1,
          # text size
          size = 3,
          # text bolding
          fontface = "bold")

```

```

# Bottom half of Figure1
# base layer ggplot
# data: salinity_prcp
salinity_plot <- ggplot(data = salinity_prcp,
  # x-axis: date
  mapping = aes(x = date,
    # y-axis: mean_salinity
    y = mean_salinity)) +
# column displaying mean salinity
geom_col(color = "grey") +
# separating points from sites into different plots
facet_wrap(~ site_full) +
# adding themeing
theme_bw() +
# x breaks by year to reduce cognitive load, labeling by
# abbreviated month and full year
scale_x_date(date_breaks = "12 months", date_labels = "%b %Y") +
# angling x axis labels to 45 degrees and to be right-aligned
theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
# labeling x and y axes
labs(x = "Date",
  y = "Monthly Average Salinity (ppt)",
  # adding title
  title = "Monthly Average NCOS Salinity w/ Precipitation Peaks",
  # adding legend title
  color = "Precipitation Events (in)") +
# adding lines from the `precipitation_peaks_facet` object
geom_vline(data = precipitation_peaks_facet,
  # oriented on the date
  aes(xintercept = date,
    # different colors by label
    color = label),
  # long dash for better viz
  linetype = "longdash",
  # line width
  linewidth = 0.75) +
# custom colors
scale_color_manual(values = wes_palette("AsteroidCity1")) +
# removing space between data and x-axis
scale_y_continuous(expand = c(0,0.1)) +
# taking out legend (redundant when stacked)
theme(legend.position = "none")

```

```
# Stacking plots
prcp_plot / salinity_plot
```

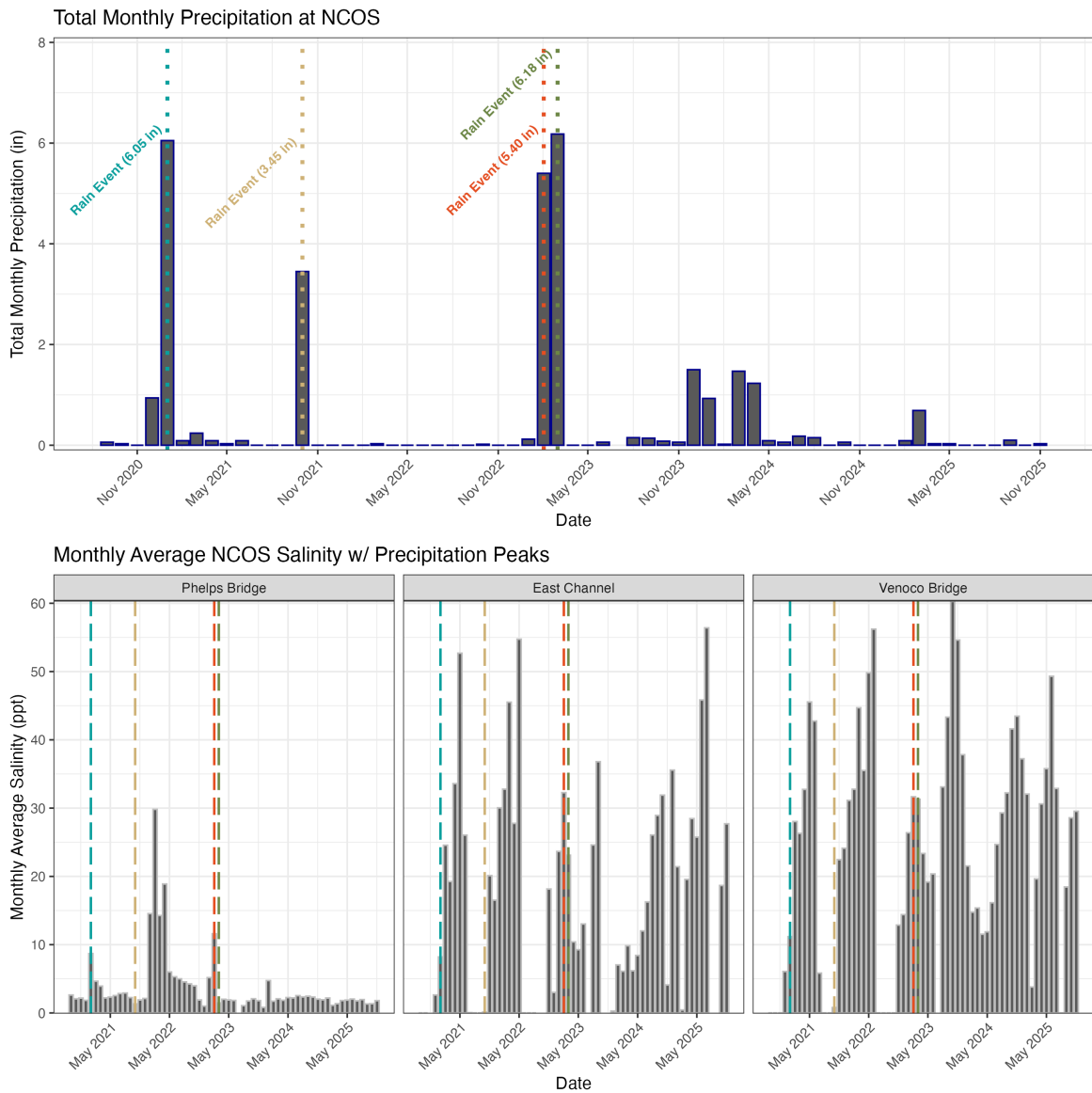


Figure 2. - How Salinity Relates to Water Surface Elevation (Venoco Bridge)

```

# base layer ggplot
# data: elevation visualization
ggplot(data = elv_viz,
       # x: average elevation
       mapping = aes(x = avg_elv,
                     # y: mean salinity
                     y = mean_salinity)) +
  # data shown by points
  geom_point(color = "maroon") +
  # each site gets own plot
  facet_wrap(~ site_full) +
  # custom color palette
  scale_color_manual(values = wes_palette("GrandBudapest1")) +
  # adding in themeing
  theme_bw() +
  # removing space between data and x-axis
  scale_y_continuous(expand = c(0,0.3)) +
  # labeling x and y axes
  labs(x = "Monthly Average Water Surface Elevation (m)",
       y = "Monthly Average Salinity (ppt)",
       # adding title
       title = "Figure 2. How Salinity Relates to Water Surface
Elevation (Venoco Bridge)")

```

Figure 2. How Salinity Relates to Water Surface Elevation (Venoco Bridge)

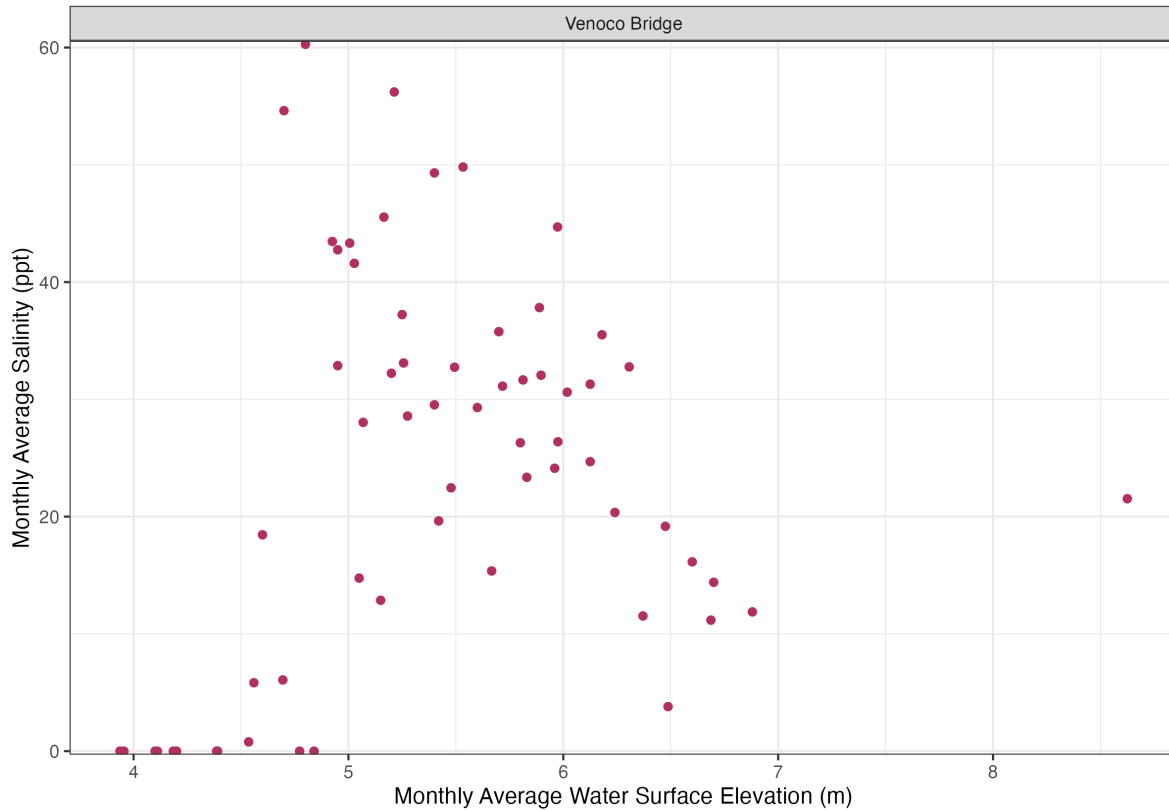


Figure 3. - How Salinity Varies Between Elevation Codes

```
# base layer: ggplot
# data: `exploratory_salinity`
ggplot(data = exploratory_salinity,
       # x: elevation code
       mapping = aes(x = elevation_code,
                     # y: mean salinity
                     y = mean_salinity)) +
# data represented by open points
geom_point(shape = 21) +
# calculating median of data & representing by points
stat_summary(fun = median,
             geom = "point",
             # custom color
```



```

        color = "maroon") +
# calculating median of data, connecting medians per site by a line
stat_summary(fun = median,
             geom = "line",
             # custom color
             color = "maroon",
             # all data into one group to connect across x categories
             group = 1) +
# labeling x and y axes
labs(x = "Elevation Code",
     y = "Mean Salinity (ppt)",
     # adding in title
     title = "Figure 3. Mean Salinity Differences Between
Elevation Codes") +
# each site has own plot
facet_wrap(~ site_full) +
# adding custom themeing
theme_bw() +
# defining the discrete elevation codes
scale_x_discrete(breaks = c(0,1, 2, 3)) +
# Moves the y-axis tick marks and labels to the right
# removing space between y axis and data
scale_y_continuous(position = "right",
                  expand = c(0,0.3)) +
# Flips the coordinate system
coord_flip()

```

Figure 3. Mean Salinity Differences Between Elevation Codes

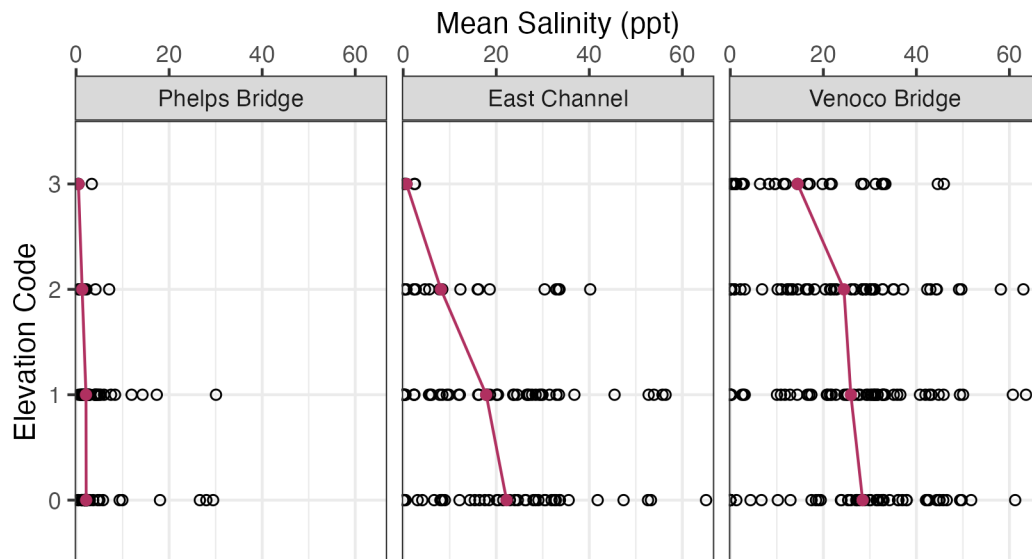
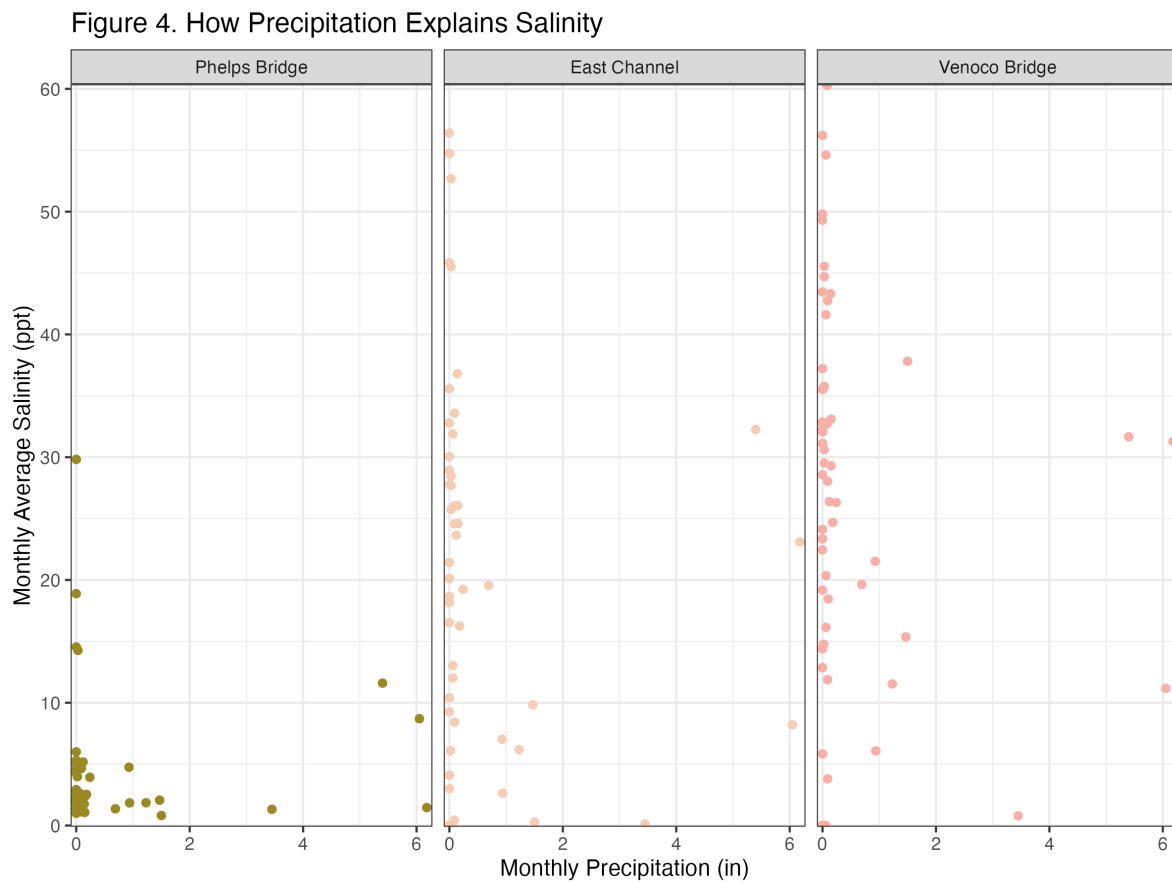


Figure 4. - How Precipitation Explains Salinity

```
# base layer: ggplot
# data: `salinity_prctp`
ggplot(data = salinity_prctp,
       # x: sum precipitation
       mapping = aes(x = sum_prctp,
                     # y: mean salinity
                     y = mean_salinity,
                     # each site own color
                     color = site_full)) +
# data represented by points
geom_point() +
# each site has own plot
facet_wrap(~ site_full) +
# custom color palette
scale_color_manual(values = wes_palette("Royal2")) +
# removing space between data and x axis
scale_y_continuous(expand = c(0,0.1)) +
# removing space between data and y axis
scale_x_continuous(expand = c(0, 0.1)) +
```

```
# custom themeing
theme_bw() +
# removing legend (redundant)
theme(legend.position = "none") +
# labeling x and y axes
labs(x = "Monthly Precipitation (in)",
     y = "Monthly Average Salinity (ppt)",
     # adding figure title
     title = "Figure 4. How Precipitation Explains Salinity")
```



Correlation Test

Correlation Test 1: Calculating Spearman's Correlation (salinity/water surface elevation)

```
# First variable: mean salinity column from the elevation
# visualization dataframe
cor.test(elv_viz$mean_salinity,
         # second variable: average water surface elevation from the
         #elevation visualization dataframe
         elv_viz$avg_elv,
         #Uses Spearman's rank correlation
         method = "spearman")
```

Spearman's rank correlation rho

```
data:  elv_viz$mean_salinity and elv_viz$avg_elv
S = 30660, p-value = 0.0748
alternative hypothesis: true rho is not equal to 0
sample estimates:
      rho
0.2279259
```

Correlation Test 2: Calculating Spearman's Correlation (salinity/precipitation)

```
# First variable: mean salinity column from the salinity_prdp dataframe
cor.test(salinity_prdp$mean_salinity,
         # Second variable: sum precipitation column from the same dataframe
         salinity_prdp$sum_prdp,
         # Uses Spearman's rank correlation
         method = "spearman")
```

Spearman's rank correlation rho

```
data:  salinity_prdp$mean_salinity and salinity_prdp$sum_prdp
S = 932653, p-value = 0.747
alternative hypothesis: true rho is not equal to 0
sample estimates:
```

rho
0.02427843

Interpreting the Results

Correlation Test 1.

A Spearman's rank correlation was conducted to examine the relationship between monthly average water surface elevation and monthly average salinity. Results indicated no statistically significant relationship between the two variables ($\rho = 0.228$, $S = 30,660$, $p = 0.075$). While the p-value narrowly exceeds the conventional significance threshold of 0.05, the moderate positive ρ suggests a weak tendency for salinity to increase with rising water surface elevation, which may warrant further investigation. These results should be interpreted with caution given the relatively small sample size, which reduces statistical power and may explain the marginal p-value. The observed trend could reflect tidal or hydrological dynamics in which higher water levels correspond to greater saltwater intrusion, though other confounding factors such as seasonal variation, site-specific conditions, or measurement timing may also contribute to this pattern.

Correlation Test 2.

A Spearman's rank correlation was conducted to examine the relationship between monthly precipitation and monthly average salinity. Results indicated no statistically significant relationship between the two variables ($\rho = 0.0242$, $S = 932,653$, $p = 0.746$). The near-zero ρ suggests that monthly precipitation has negligible association with average salinity at this scale. It should be noted that exact p-values could not be computed due to ties in the data, so a normal approximation was used; this does not meaningfully affect the interpretation. These results may reflect lag effects between precipitation events and salinity response, spatial or hydrological disconnect between measurement sites, or the influence of other drivers, such as evaporation or tidal exchange, that operate independently of local precipitation.

4. Other exploratory visualizations

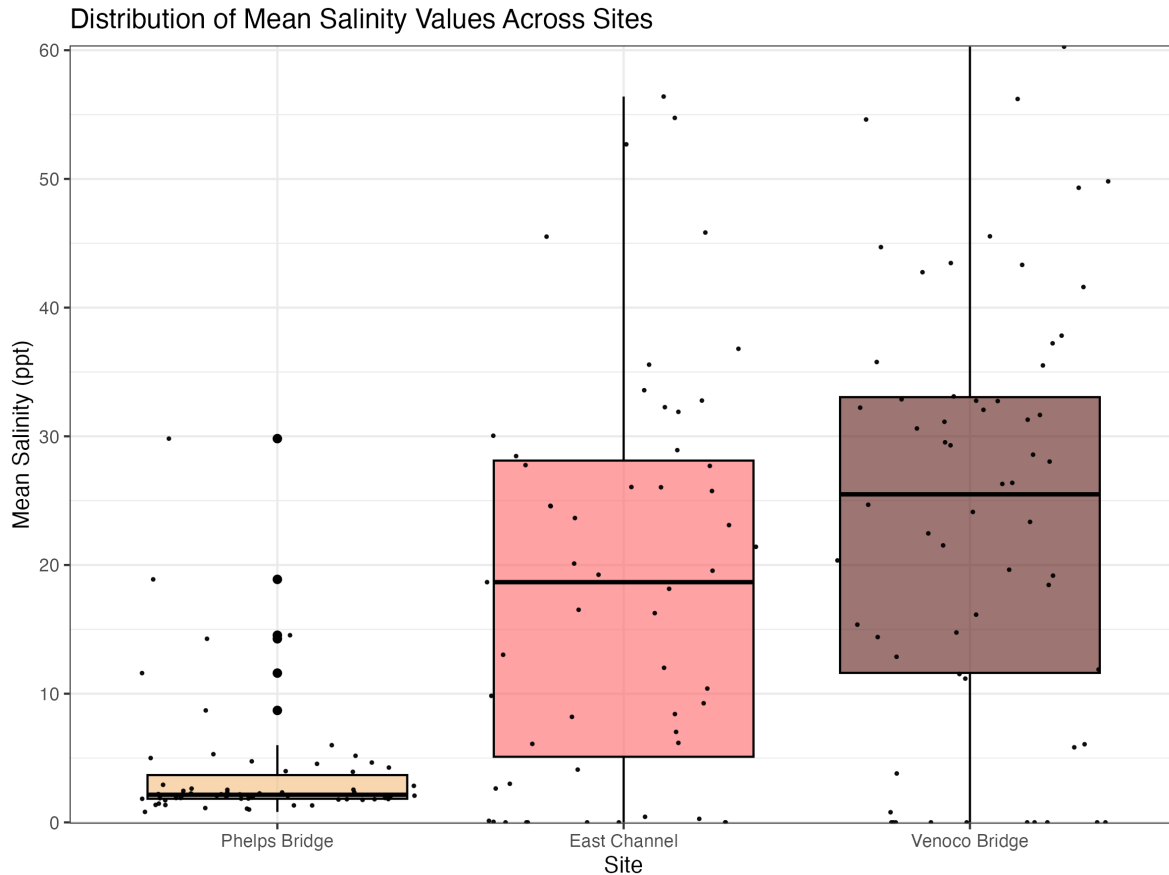
Comparing Across Groups

Boxplot w/ Jitter - Distribution of Mean Salinity Values Across Sites

```

# base layer: ggplot
# data: salinity~prcp
ggplot(data = salinity_prcp,
       # x: site name, y: mean salinity, coloring in by site
       aes(x= site_full, y=mean_salinity, fill = site_full)) +
# boxplot to display data
geom_boxplot(color = "black") +
# adding jitter to points for better visualization
geom_jitter(color="black", size=0.4, alpha=0.9, height = 0) +
# custom coloring and a 0.6 transparency to visualize data points
scale_fill_manual(values = alpha(wes_palette("GrandBudapest1"), 0.6)) +
# removing space between data and x axis
scale_y_continuous(expand = c(0, 0.1)) +
# adding in a theme
theme_bw() +
# taking out legend
theme(legend.position="none") +
# labeling x and y axes
labs(x = "Site",
     y = "Mean Salinity (ppt)",
     # adding in a title
     title = "Distribution of Mean Salinity Values Across Sites")

```



- The addition of this plot was taken as a suggestion made by An after reviewing our Work Plan/Project Proposal. This figure aims to demonstrate that not only does the elevation code of a site yield a difference in monthly mean salinity (as seen in figure 3), but also that the three NCOS sites also differ in their overall monthly mean salinity values. This plot sets the basic understanding of salinity variety across sites while also adding context to one of the visualizations directly answering our question.

Distribution of Observations

of Observations per Site

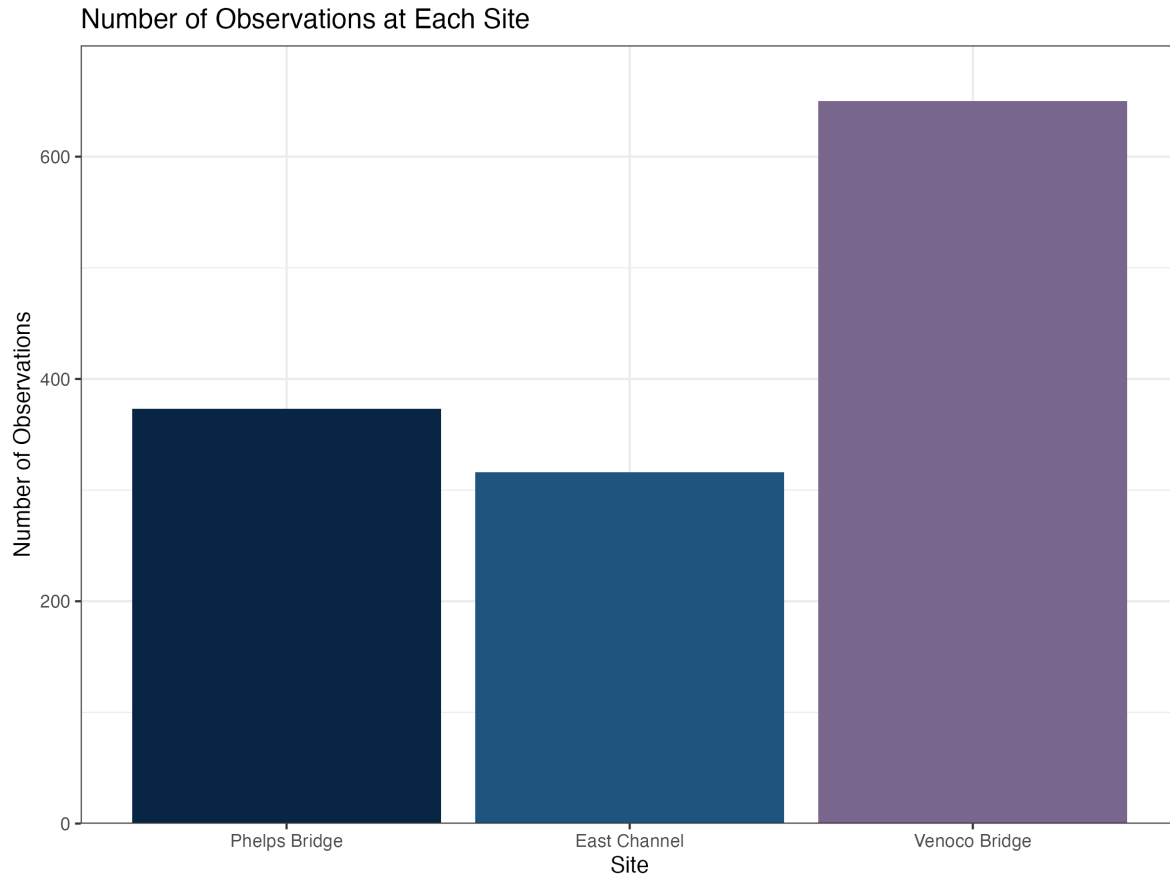
```
# base layer: ggplot
# data: water_quality_weather, filtered and releveled inline
ggplot(data = water_quality_weather |>
  # removing outlier IDs, "other" sites, and keeping only
```

```

#elevation codes 0, 1, and 2
filter(!global_id %in% salinity_outlier_ids,
       site_name != "other",
       elevation_code %in% c("0", "1", "2")) |>
# setting site order from lowest to highest average salinity
mutate(site_full = fct_relevel(site_full,
                              "Phelps Bridge",
                              "East Channel",
                              "Venoco Bridge")),

# x axis: site name
mapping = aes(x = site_full)) +
# custom fill colors
scale_fill_manual(values = natparks.pals("Volcanoes")) +
# removing the space between data and x axis
scale_y_continuous(expand = c(0, 0.1),
                   # definign limits to include all data
                   limits = c(0, 700)) +
# bar chart with fill colored by site
geom_bar(aes(fill = site_full)) +
# adding theming
theme_bw() +
# removing legend (redundant)
theme(legend.position = "none") +
# labeling x and y axes
labs(x = "Site",
     y = "Number of Observations",
     # adding title
     title = "Number of Observations at Each Site")

```

of Observations per Elevation Code

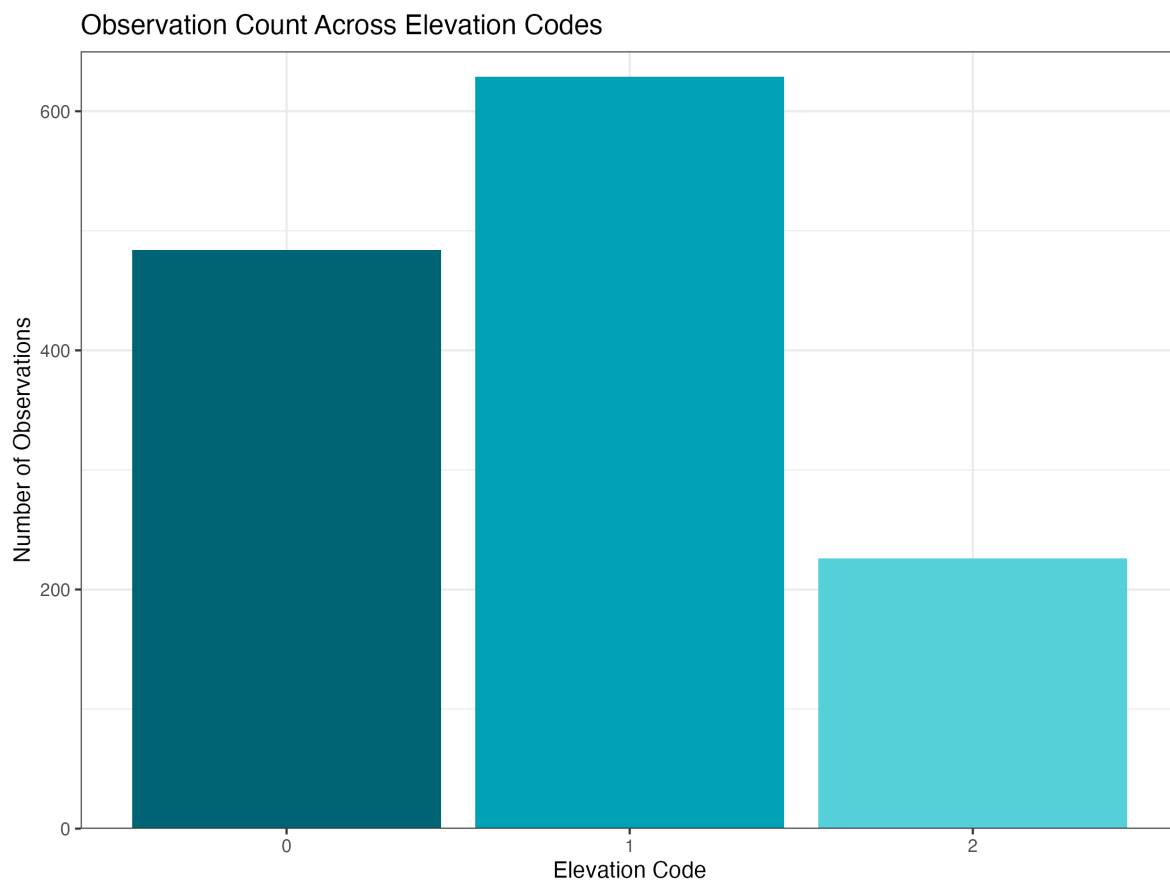
```
# base layer: ggplot
# data: water_quality_weather, filtered inline
ggplot(data = water_quality_weather |>
  # removing outlier IDs, "other" sites, and keeping only
  #elevation codes 0, 1, and 2
  filter(!global_id %in% salinity_outlier_ids,
    site_name != "other",
    elevation_code %in% c("0", "1", "2")),
  # x axis: elevation code
  mapping = aes(x = elevation_code)) +
# bar chart with fill colored by elevation code
geom_bar(aes(fill = elevation_code)) +
# custom fill colors
```

```

scale_fill_manual(values = natparks.pals("Banff")) +
# removing space between data and x axis
scale_y_continuous(expand = c(0, 0.1),
                    # defining vertical limits
                    limits = c(0, 650)) +

# adding theming
theme_bw() +
# removing legend (redundant)
theme(legend.position = "none") +
# labeling x and y axes
labs(x = "Elevation Code",
     y = "Number of Observations",
     # adding title
     title = "Observation Count Across Elevation Codes")

```



Both of these figures were added last-minute to the work plan after our data visualization presentation in class. We feel that the addition of this information is important in contextualizing

our outputs. Venoco Bridge has the most amount of observations, followed by Phelps Bridge, and finally, East Channel. We can also see that elevation code 1 has the most amount of observations, followed by 0, and finally, 2. Understanding the distribution of observations allows for the appropriate selection of statistical methods, correct interpretation of analytical results, and effective and accurate communication of findings.

5. Plan for Elective

For our elective, we have chosen to create a map of the NCOS area depicting the different keystone species that breed, live, and migrate through these areas. The map will depict water levels from spring of 2023, and be modeled after other NCOS maps online. The map will specifically point to locations such as Phelps Bridge, East Channel, and Venoco Bridge, representing our three main points in our data analysis process. At each site I will choose pictures of different keystone organisms that live in these areas, as well as their preferred salinity conditions according to outside local research. The map will be made of collaged paper materials, and will have additional pictures of the organisms that live and exist there. For each organism, there will be their ideal salinity condition (shown through cups of salt or another visual model) next to the organism, to depict the conditions they prefer to live in. The description of the image will be one map that is multimedia and will be handmade and depict a map. Over the next two weeks, our group will collect the adequate materials in the next week, print out an example map to add our collage materials over it, look at water levels for the 2023 year and adjust as necessary, and research organisms specific to the NCOS site. Before the end of the week 9 weekend, we will have all materials collected and information logged on a shared google doc so that all parties can contribute and see the research and materials needed. Over the week 9 weekend, group members will select a time to meet up and work artistically on the project, and finishing touches will be made in the remaining time before it is due, including working on a presentation format.